MLOPS / WEEK 3 / LECTURE 05

Model serving

Observable deployment

From a trained artifact to a service we can operate, inspect and trust.

TRAN Cong Minh

RUNTIME RECORD

REQUEST

weather + calendar

MODEL

baseline-v1

SERVICE

code + configuration

PREDICTION + EVIDENCE

A model is not a service

Training creates an artifact. Serving adds an operating contract.

ARTIFACT

model.pkl

A file exists.
Nothing is operating yet.

+

THE SERVICE AROUND IT

Reachable
A caller can use it.

Repeatable
The same contract holds.

Inspectable
Identity and failure evidence.

Ask what happens when traffic spikes, features change or the wrong artifact loads.

Three decisions to defend

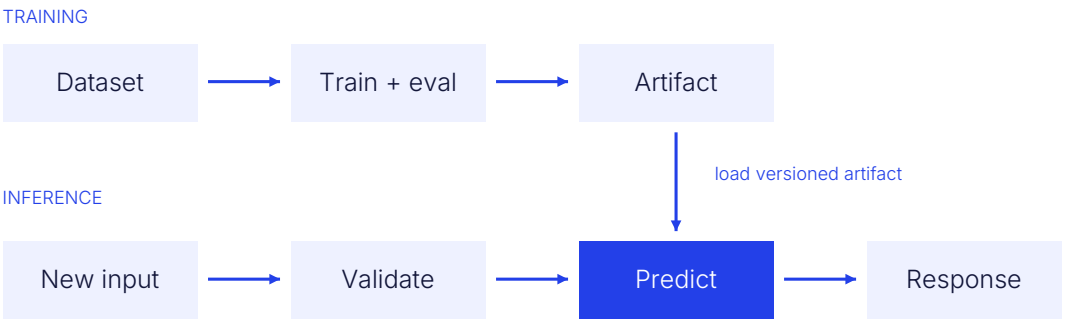
Tie each choice to system behavior and inspectable evidence.

Serving pattern	Match latency, freshness and input arrival.
Execution location	Justify network, hardware and operating cost.
Operational evidence	Prove readiness, identity and failure behavior.

Decide, deploy and verify: each choice needs evidence you can inspect.

Train offline. Serve new inputs.

The artifact crosses the boundary; the training session does not.



Load the artifact into the runtime; keep feature semantics compatible with training.

Every prediction needs a contract

A valid HTTP response does not prove that the intended model ran.

INPUT

Features

Schema, units, order, time and missing-value rules.

COMPUTE

Artifact + runtime

Immutable identity, exact preprocessing and compatible dependencies.

OUTPUT

Prediction + identity

A defined response schema and a traceable request and model.

Lecture 4 still applies: validate requests and test the containerized API.

Latency is the whole path

Network and queue time can dominate a fast model.

REQUEST → RESPONSE / SCHEMATIC, NOT MEASURED



End-to-end latency is the sum of every stage.

Optimize the measured bottleneck. Inspect tail latency (for example p95), not only the mean.

01

Choose the serving pattern

Batch, online, asynchronous and hybrid are patterns—not a ranking from bad to good.

Keep three decisions separate

Computation, interaction and location describe different axes.

Computation	Batch / per-request
Interaction	Synchronous / asynchronous
Location	Cloud / self-hosted / edge

Batch can run in the cloud. Per-request inference can run on a device.

Urgency × input arrival

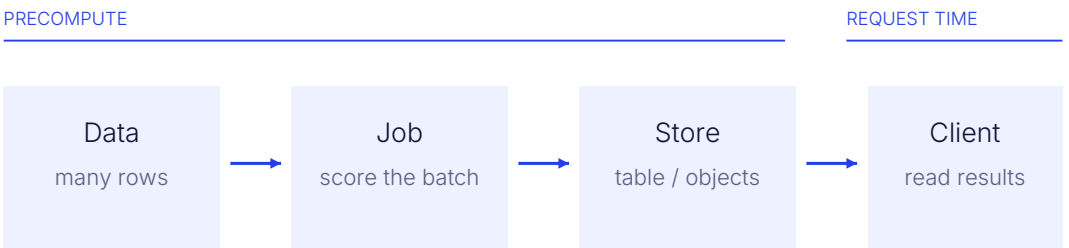
A decision aid, not a strict taxonomy: real systems combine patterns.

	RESULT DEADLINE: Immediate	RESULT DEADLINE: Can wait
Single input	Online / sync	Async request
Grouped inputs	Often a poor fit	Batch job

Classify the requirement before choosing infrastructure.

Batch: compute first, read later

Materialize predictions before the client asks for them.



A fast lookup may only mean the expensive inference already happened.

The trigger sets freshness

Batch means grouped work—not necessarily a nightly schedule.

SCHEDULED

Periodic refresh

Every day, every week or a defined production window.

EVENT DRIVEN

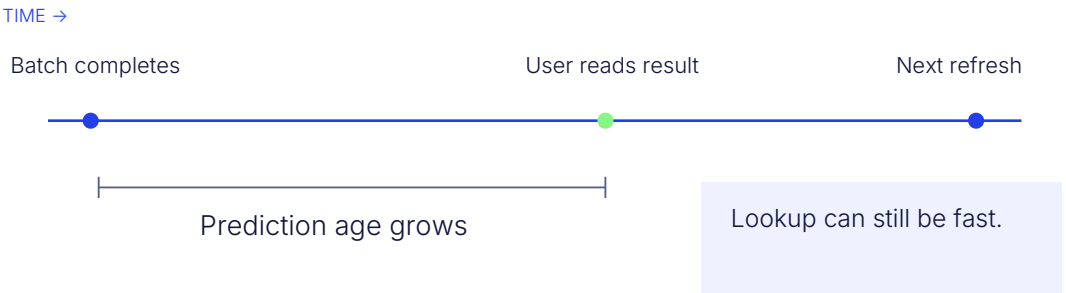
New data is ready

A file lands or a complete data partition becomes available.

Examples: inventory forecasts, document scoring and sentiment over a dataset.

Fast lookup ≠ fresh prediction

Batch moves inference latency out of the user's request.



Unseen inputs need another batch run or a separate online path.

Batch trades freshness for throughput

Precomputed results are convenient, not inherently real-time.

BENEFIT

Amortize the work

Large datasets; predictable windows; fast lookup; less request-time serving burden.

COST

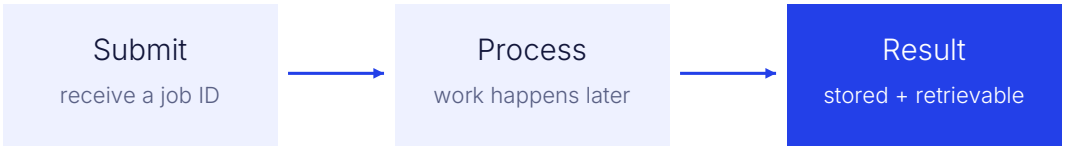
Results age

Storage and orchestration; no immediate answer for unseen inputs; a failed job delays refresh.

New inputs can be scored in a batch once they are supplied to that batch.

Async: submit now, collect later

The caller does not hold a request open until inference completes.



COMPLETION

Push: notify the caller

Pull: retrieve by ID

Batch is often async. One large input, such as an audio file, can also be async.

Batch and async are not synonyms

Grouping work and waiting for results are independent decisions.

Batch + async	Nightly scoring pipeline
Single + async	One long audio transcription
Single + sync	Fraud score during checkout

Separating the axes prevents the wrong infrastructure decision.

Online: compute for each request

The caller needs a result within a useful response deadline.



Features must exist at request time. Online inference does not require the cloud.

Features matter as much as weights

Production needs a compatible equivalent of training-time preparation.

TRAINING

Historical observations

Temperature, calendar and holiday columns; chronological train/validation/test boundaries.

SERVING

Available at request time

Same feature names, units and semantics; an explicit missing-value policy.

A successful API call does not prove that training and serving features mean the same thing.

Fresh answers need reliable operations

Online serving adds a latency, availability and capacity burden.

LATENCY

Meet the deadline

Budget for network, queues, feature work and inference.

AVAILABILITY

Handle failure

Plan restarts, warm-up, readiness and dependency failures.

CAPACITY

Measure the limit

Load-test request rate, parallelism, CPU and model memory.

Not every online workload needs a GPU. Provision from measurements.

Online ≠ cloud

The response deadline and execution location answer different questions.

ONLINE + CLOUD

Web prediction API

A remote service answers a browser or mobile client.

ONLINE + EDGE

Local device model

A robot or phone answers without a network round trip.

Both can meet a real-time deadline—and both can fail on features, compute or identity.

Match the pattern to the requirement

Choose the simplest design that meets latency, freshness and scale.

DIMENSION	BATCH	ONLINE	ASYNC
Trigger	schedule / event	new request	submit a job
Result	stored output	within deadline	later
Priority	throughput / cost	tail latency	long work
Example	forecast refresh	spam decision	audio job

Async is an interaction pattern; it can coexist with batch or per-input computation.

A new bike-demand requirement

A hypothetical extension of Bike Demand.

DISCUSS / CHOOSE + JUSTIFY

Predict tomorrow's hourly demand each morning from a weather forecast.

A Online

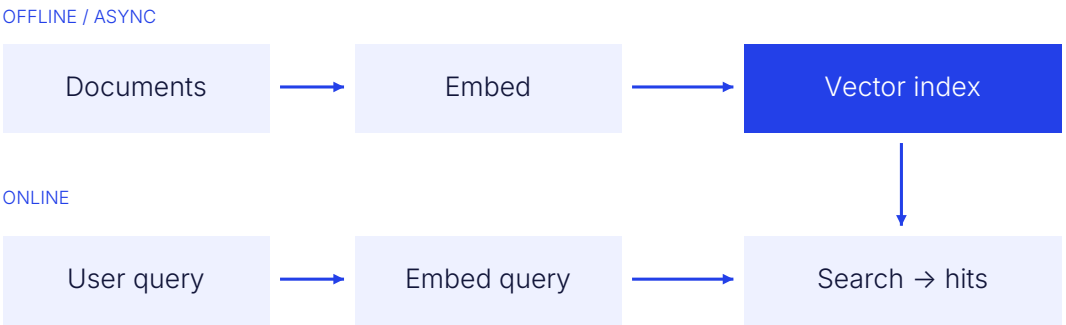
B Batch

C Streaming

Defend the choice using freshness, deadline and how many inputs arrive together.

Hybrid retrieval has two paths

Prepare document vectors offline; embed each query online.



The index is a stored dependency, updated asynchronously as documents arrive.

Precompute common work. Rank live.

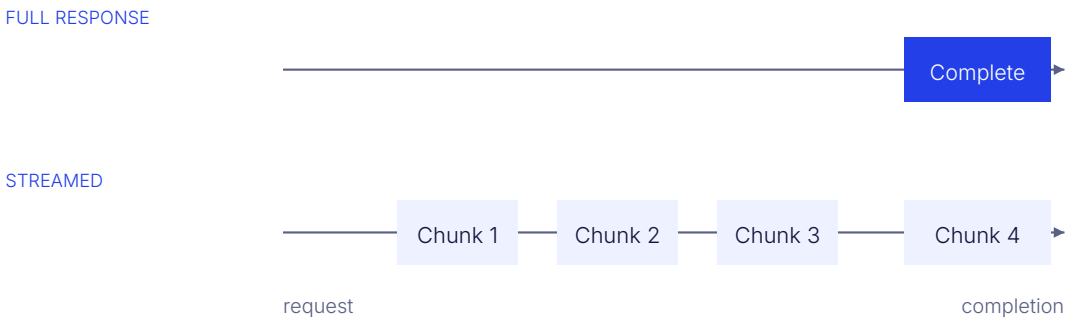
A hybrid recommender separates reusable work from request-specific work.



Cache what is reusable; compute what depends on this request. Example: content rows.

Streaming reveals output sooner

Chunks can reach the user before the complete response is generated.



Perceived responsiveness can improve without reducing total compute time. Timing is schematic.

Streaming does not require WebSockets

Choose a transport for the direction of communication you need.

HTTP STREAMING / SSE



Incremental one-way output; often enough for text generation.

WEBSOCKET



Persistent two-way exchange for interactive applications.

REST is an architectural style, not the strict opposite of a WebSocket.

One app can combine patterns

Treat each stage as a separate requirement—not a product-wide label.

Can wait	Scheduled bike-demand forecast → batch
Needs fresh input	Current weather query → online
Takes a long time	Submit work → job ID → collect later

Write down the response deadline, prediction freshness and input arrival pattern.

02

Where should it run?

Placement adds network, hardware, privacy and operating constraints to the serving pattern.

Place the model near its constraints

Cloud, self-hosted and edge describe where the runtime lives.

REMOTE

Public cloud

Provider infrastructure; managed services or your own containers.

YOUR INFRA

Self-hosted

Your organization operates the servers and deployment platform.

ON DEVICE

Edge

Inference near the user or sensor, possibly without connectivity.

On-premises usually means organization-run infrastructure; phone/robot inference is on-device.

No location wins every trade-off

Compare the constraints that dominate your actual workload.

CONSTRAINT	REMOTE CLOUD	LOCAL / SELF-HOSTED
Network	remote connection	may work disconnected
Latency	includes network hop	depends on local compute
Operations	provider services	own server/device lifecycle
Scale	elastic options	provision the fleet
Data control	region + contracts	local if designed that way

Privacy and residency require a concrete security/legal design—not just a location label.

Place each component separately

An illustrative split for smart-glasses translation—not a required architecture.

GLASSES

Sense + display

Capture the sign and maintain a tight motion-feedback loop.

PHONE

Local processing

OCR or a lightweight translation model/cache, when feasible.

CLOUD

Heavy work

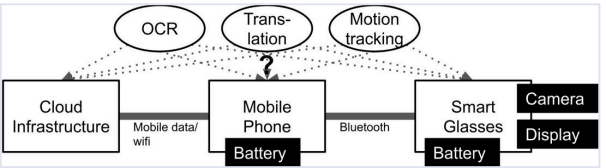
A larger translation model or updates when a connection exists.

Which functions must survive when the phone loses connectivity?

Real case: which part runs where?

Smart glasses sense, a phone relays, the cloud can translate. Each step has its own place.





Decide per component: what must run on the device, what can wait for the network.

Partition an AR workload

Decide where OCR, translation and motion tracking should run.

DISCUSS / CHOOSE + JUSTIFY

A user points smart glasses at a sign. The translation must appear without lag.

A All cloud

B All device

C Split pipeline

Consider motion feedback, model size, battery, uplink quality and update frequency.

Offline inference is still production

Downloaded language packs can support some translation features offline.

ON-DEVICE MODEL

Bonjour

Hello

No network round trip.

Installed languages, model availability and quality vary. Updates may still require connectivity.

Feature support depends on the app, platform and language; do not generalize to every function.

A model that lives on the phone

Offline translation downloads a language model to the device.





No network at use time, but the model on the device still needs updates and checks.

Seven questions before choosing a host

Requirements first. Product catalog second.

Input size
What crosses the network?

Output size
Do results need streaming?

Compute
CPU, GPU, memory, energy?

Latency
What delay is acceptable?

Artifact
Size and update cadence?

Operating cost
Compute, storage, transfer?

Connectivity
What if the network fails?

Also assign ownership for compliance, security, maintenance and recovery.

Local removes a hop—not the risks

Locality can help control and connectivity; it does not guarantee security.

WHY LOCAL

Work without uplink

Fewer WAN dependencies; control device-side data flow; meet local feedback deadlines.

WHAT YOU OWN

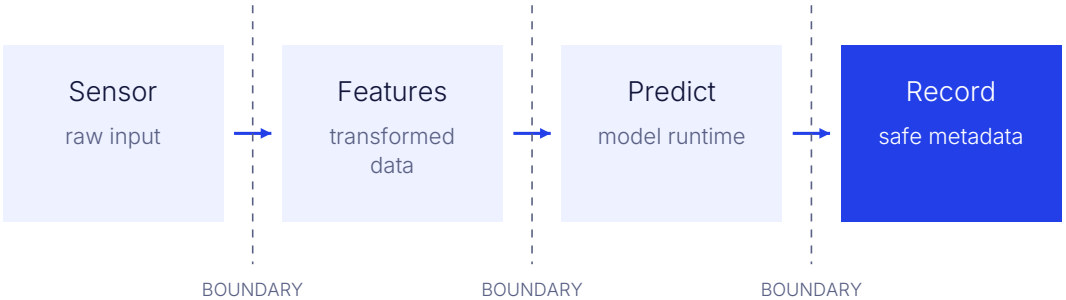
The device lifecycle

Patching, secure storage, physical access, theft and safe model rollout across the fleet.

GDPR does not categorically ban third-country transfers; compliance depends on safeguards.

Map data flows, not privacy slogans

Inspect collection, transformation, transit, storage, logs and retention.



What leaves each boundary? Keep raw payloads and personal data out of routine logs.

Justify the execution location

Use the few constraints that actually dominate the application.

DISCUSS / CHOOSE + JUSTIFY

A combine harvester must adjust its operation even when the rural network is unavailable.

A Cloud-only

B Edge-first

C Nightly batch

Dominant constraints: local feedback latency and intermittent connectivity.

03

Who operates the service?

Custom containers, managed platforms and serverless shift work—not accountability.

Custom vs managed: who owns what?

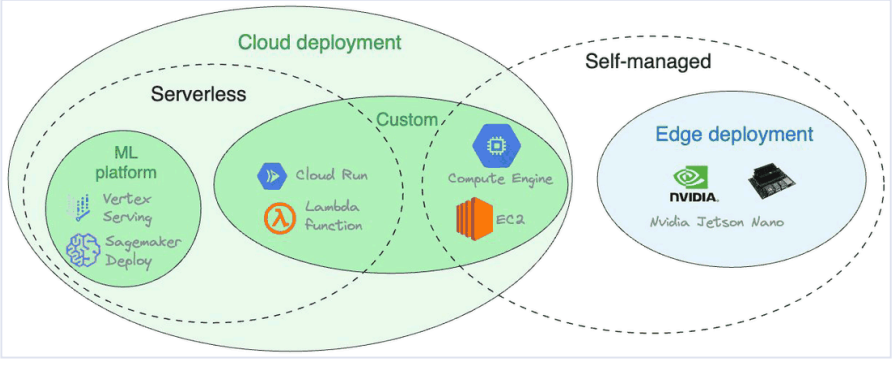
Compare engineering effort, control and constraints—not only instance price.

RESPONSIBILITY	CUSTOM SERVICE	MANAGED ML
Model + contract	your team	your team
App packaging	your API / image	platform-compatible artifact
Serving lifecycle	you configure it	provider handles more
Scaling / infra	you or host tools	platform controls + limits

Managed is not always inflexible; custom is not always cheaper. Compare the workload.

Cloud, serverless, platform, edge

One map of hosting options; you pick how much of the stack you own.



More managed means less to operate and less control over the environment.

Serverless is not server-free

The provider takes more provisioning and scaling work.

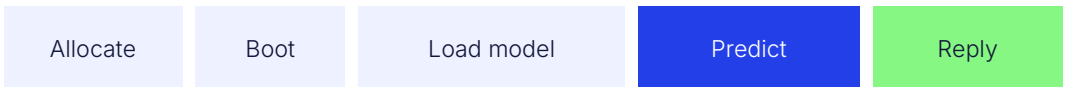
APPLICATION	Your handler or container
PLATFORM	Start, route and scale
INFRASTRUCTURE	Compute and networking

Functions and container services are common interfaces. Security and correctness remain yours.

Cold starts add work before inference

A new instance may initialize the runtime and load the model first.

COLD REQUEST



WARM REQUEST



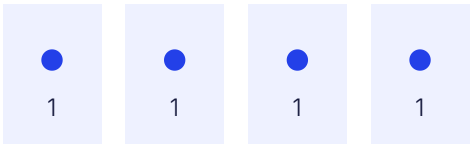
Already initialized.
No initialization delay on this path.

Schematic only. Measure cold/warm paths; consider warm instances and lean initialization.

Concurrency is not infinite capacity

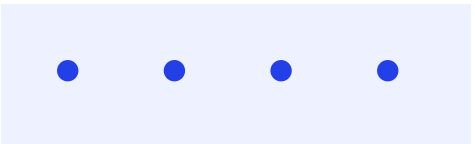
More in-flight requests may save instances—or overload one.

FOUR INSTANCES × ONE REQUEST



More model copies

ONE INSTANCE × FOUR REQUESTS

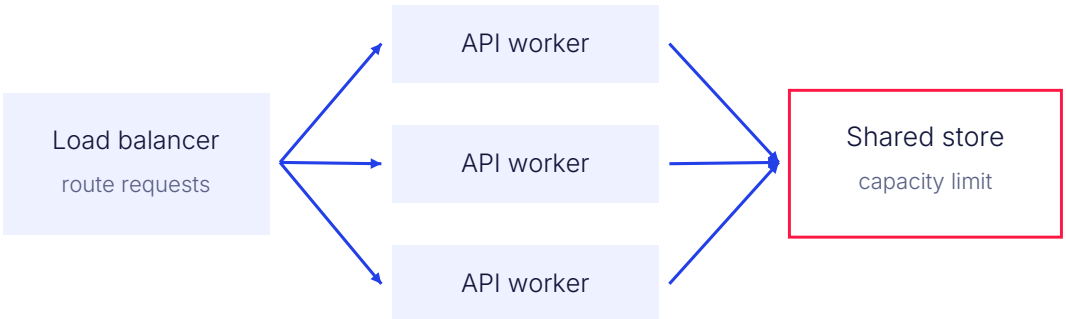


More shared contention

Illustrative. Load-test throughput, tail latency and memory together.

Scale the bottleneck you actually have

Replication helps only if dependencies and parallelism can support it.



Watch shared stores, model memory per worker, startup delay and maximum instance count.

One container. Different hosts.

Separate the runtime contract from the host's product category.

Server / VM	You operate the application lifecycle.
Container platform	Platform runs replicated containers.
Managed ML	Platform manages prediction endpoints.
Edge / embedded	The device runs the model near its sensor.

These are categories—not interchangeable products with identical contracts.

Costs have three different drivers

Running resources, retained data and network traffic all create bills.

COMPUTE

Runtime

CPU/GPU, memory, idle capacity and request duration.

STORAGE

Artifacts + data

Images, model artifacts, databases, object storage and retained logs.

TRANSFER

Networking

Egress, load balancing, requests and cross-region traffic.

Prices vary by region, service and billing mode. There is no universally cheapest host.

Spending limits are operating controls

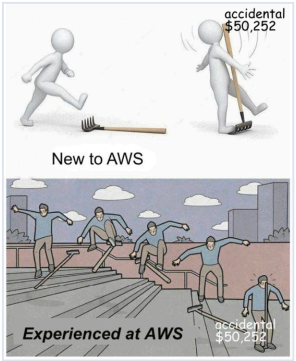
Plan shutdown, scaling, budgets and alerts before leaving resources running.

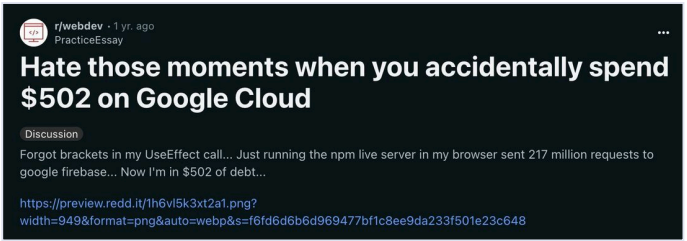
Stop idle work	Avoid permanent experimental VMs.
Cap scaling	Know instance limits and burst behavior.
Watch usage	Set budget alerts and review dashboards.
Right-size	Measure utilization and model needs.
Batch if viable	Amortize work when freshness allows.

A budget alert is not necessarily a hard spending cap. Check platform-specific controls.

Cloud bills surprise teams

Public accounts of accidental spend show why budgets are an operating control.





Set a budget, an alert and a quota before you deploy anything.

Managed hosting is still your system

You own correct inputs, identity, outputs, access and failure evidence.

DISCUSS / CHOOSE + JUSTIFY

A managed endpoint returns 200 OK but loads yesterday's model. What must happen?

A Blame provider

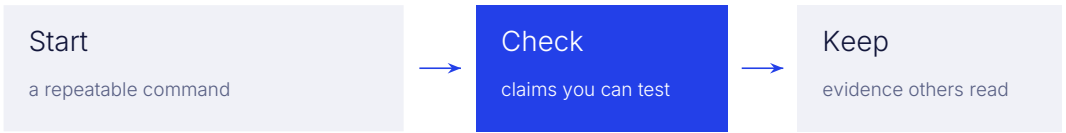
B Accept the result

C Detect mismatch

Check runtime artifact identity end to end; hosting alone cannot guarantee correctness.

Start it. Then verify it.

“Container started” is not “deployed”.



`Running is not the same as verified.`

Deployed means repeatable, checked and evidenced.

Two things ship: image and model

They are different artifacts with different identities.

THE IMAGE

Code and its environment

- Built from a commit
- Identified by an image ID
- Changes when the code changes

THE MODEL

A trained artifact

- Produced by training
- Identified by a version and a hash
- Changes when you retrain

How a model reaches the container (inside the image, or supplied from outside) decides what a model update costs.

Configuration enters from outside

The image is the same everywhere; the settings are not.

WHY

Same image everywhere

Laptop, CI and deployment run one image with different settings.

HOW

Set at run time

Pass values when you start it; do not bake them in.

NEVER

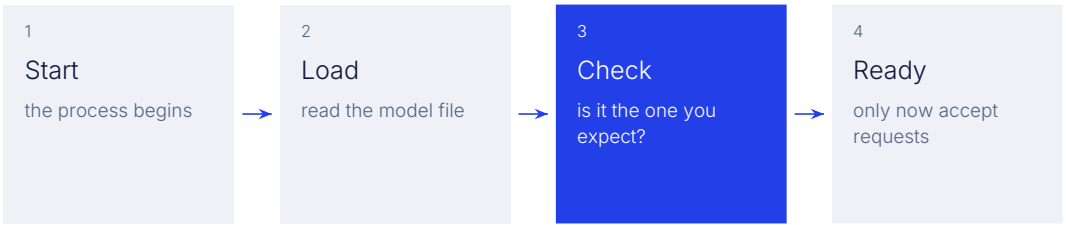
Secrets in the image

Anything baked into an image can be read by whoever holds it.

Decide which settings exist, where they come from, and what happens when one is missing.

Load, check, then become ready

A service should say it is ready only when it can serve correctly.



`If the check fails: say so, stay not ready, serve no guesses.`

Fail closed: an honest 503 is better than a confident wrong answer.

04

Make it observable

“It responds” is not the same as “we can explain what happened”.

Three questions an operator must answer

Status and model version are clues—not a complete explanation.

HEALTH

Is it healthy?

Rate, errors, latency, saturation and restarts.

FAILURE

What failed?

Correlate validation, dependency and prediction failures.

IDENTITY

What ran?

Identify code, model artifact, configuration and rollout.

Start with inspectable logs and model-version checks—not a full monitoring platform.

Logs, metrics and traces fit together

Schematic views of the same request path—not measured data.

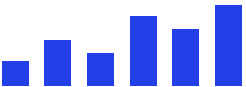
LOGS



What happened?

Events, request ID, status, version.

METRICS



How often? How much?

Rate, errors, p95, utilization.

TRACES



Where did time go?

Linked spans across components.

OpenTelemetry supports all three, but you do not need it to begin.

Structured logs make failures queryable

Consistent fields let operators search events without parsing prose.

ILLUSTRATIVE JSON / TOY SERVICE

```
{
  "event": "prediction_completed",
  "request_id": "req-demo-17",
  "status": "ok", "duration_ms": 42,
  "model_version": "baseline-v1",
  "service_revision": "demo-rev-b",
  "timestamp": "2026-10-01T10:00:00Z"
}
```

CORRELATE

request_id

EXPLAIN

event, status, duration

IDENTIFY

model + service

Illustrative values from a toy service. Which fields you log is your decision.

One request, one event

What a log event must let an operator reconstruct.

The operator asks	The event carries
Which request was it?	a correlation id
What happened?	route, status, outcome
How long did it take?	a duration
Which model and which code?	identity fields
Is anything sensitive?	only what a stated purpose needs

The previous slide is a toy service; your event design is yours.

Log metadata—not private payloads

Operational usefulness should not depend on collecting sensitive data.

PREFER

Operational metadata

Request ID, route, status, latency, model identity and service revision.

AVOID BY DEFAULT

Sensitive payloads

Tokens, credentials, raw personal data and complete uploaded documents.

Redact and minimize. Define retention, access controls and applicable compliance requirements.

Request, model and code are different IDs

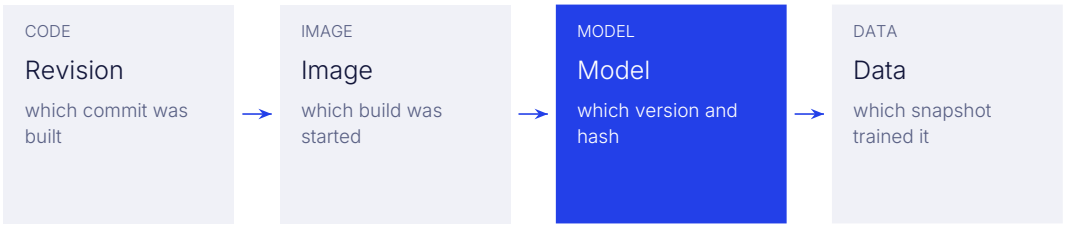
Changing the service need not change the model—and vice versa.

request_id	One prediction attempt; correlate its events.
model_version	Serialized artifact + preprocessing identity.
service_revision	Code/image revision hosting the request.

Identifiers must be stable and tied to real artifacts—not convenient filenames.

The identity chain on the wire

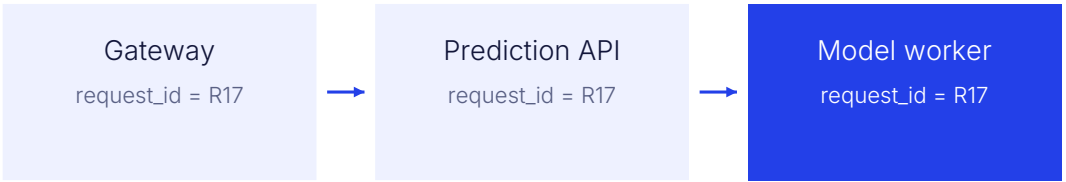
Each link answers one question about what ran.



Evidence should name the links it relies on; a missing link is a gap.

Follow one request through the system

Preserve the correlation key as work crosses component boundaries.



R17 / one correlation key across all three components

Trace/span IDs can add hierarchy. A request ID is a useful first step.

Alive does not mean ready

The process can accept connections before the model is usable.

LIVENESS

Is the process running?

Use a lightweight probe to decide whether a restart is needed. Avoid inference on every probe.

READINESS

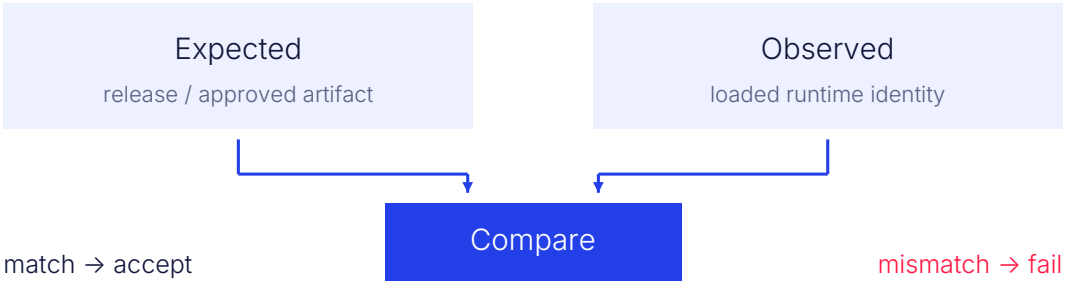
Can it serve correctly?

Model loaded, required dependencies available and necessary checks satisfied.

A readiness probe should return 200 only when a usable model is loaded.

Assert the intended artifact is loaded

Compare an independent expectation with the observed runtime identity.



A numeric prediction is not enough. Verify identity against the actual API contract.

Start with four service signals

Each measurement should answer a concrete operating question.

Request rate

How much traffic arrives?

Error rate

How often does serving fail?

Latency

How long do callers wait?

Saturation

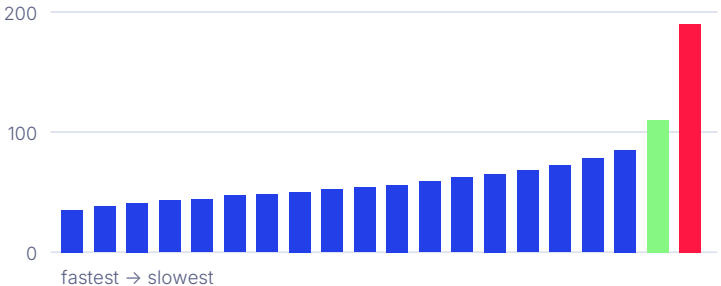
Where is capacity constrained?

Assess data quality, drift and model outcomes later—when trustworthy ground truth exists.

Average latency hides the tail

p95 describes the slow end—not the typical request alone.

20 SORTED REQUESTS / ILLUSTRATIVE MILLISECONDS



110 ms

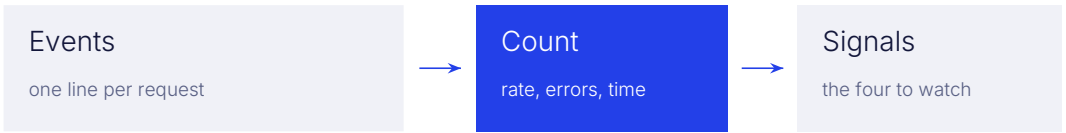
NEAREST-RANK p95

19 of 20 requests are at or below this value.

Nearest-rank p95 is the 19th value in this 20-request example. No benchmark is claimed.

From logs to signals

Your own log lines are enough to start measuring.




```
No new tool is needed to begin.
```

Lecture 8 builds on this when you read signals during an incident.

200 OK is not proof of the right model

The request can succeed while the deployment is wrong.

HTTP STATUS

200

Request succeeded

ARTIFACT IDENTITY

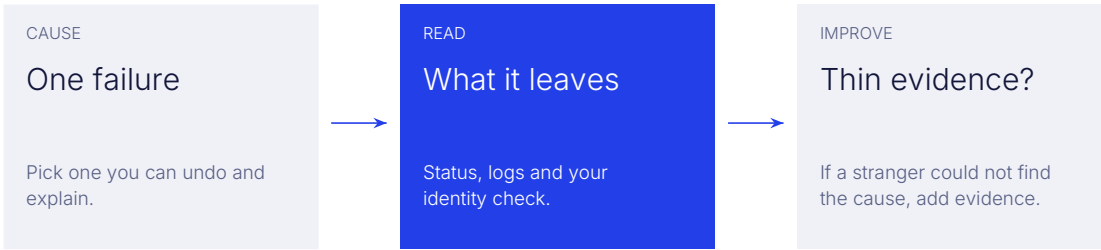
Expected	v2
Observed	v1

Deployment check must fail.

Inspect the loaded artifact identity and assert it in an external verification check.

Make it fail on purpose

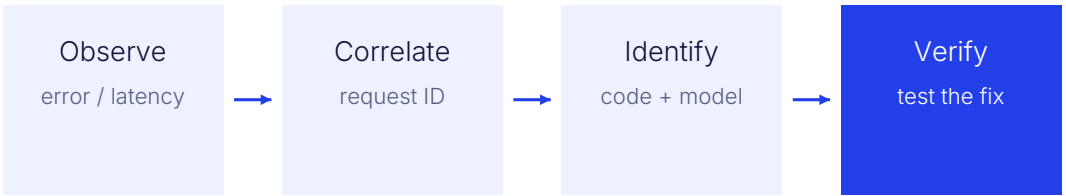
A deployment is trusted after you have seen it fail in a way you understand.



Record one failure and what it left behind.

Debug from symptom to evidence

Follow the trail instead of redeploying blindly.



Lecture 8 returns to this with incident recovery and rollback verification.

Five claims before “deployed”

Five claims to check from outside. How would you test each, and what would you keep?

Starts	Listens and loads the required artifact.
Serves	A documented valid input returns a valid prediction.
Rejects	Invalid input produces a predictable, safe error.
Identifies	Reports the intended model and code identity.
Explains	Leaves useful, non-sensitive structured logs.

Only claim a passed check when actual commands and observed output support it.

05

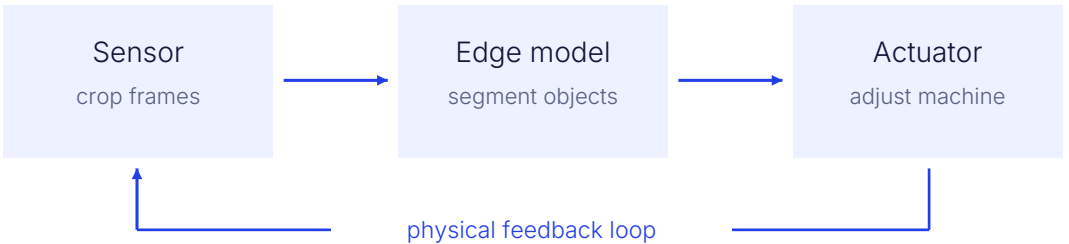
Architecture in context

Serving design is a set of justified trade-offs backed by measurable evidence.

PATTERN / PLACEMENT / OPERATIONS / EVIDENCE

A field machine needs a local loop

Segment the crop stream and adjust equipment without a cloud round trip.



Measure end-to-end control latency. Power, model size and disconnected operation constrain design.

A field machine cannot count on the network

A combine harvester must adjust itself even where there is no signal.



Local inference keeps working; updates, logs and monitoring must tolerate gaps.

Measure before compressing the model

Smaller models and quantization trade quality, speed and compatibility.

1 Profile	Locate the measured bottleneck.
2 Simplify	Try smaller architecture or input resolution.
3 Optimize	Quantize or prune where valid.
4 Deploy	Use a compatible accelerated runtime.
5 Verify	Recheck quality and actual end-to-end latency.

Example runtime: TensorFlow Lite. Hardware support and performance must be verified.

An audio archive is a batch workload

Prioritize throughput, completion quality and operating cost.

THOUSANDS OF RECORDINGS



Whisper and other speech-recognition models are candidates—not automatic performance wins.

Optimize useful throughput per cost

Changing the ASR model alone does not guarantee a speedup.

MODEL CHOICE

Benchmark candidates

Word error rate, language coverage, runtime and memory on representative recordings.

SYSTEM DESIGN

Reduce wasted work

Tune batch size and scheduling; inspect utilization; isolate failures and control retries.

A different wav2vec2 variant is not automatically faster. Compare quality and speed empirically.

Different workloads. Different evidence.

Measure the requirement that actually matters to the application.

DESIGN AXIS	HARVESTER	AUDIO ARCHIVE
Serving	online / synchronous	batch / async
Location	edge-first	cloud / self-hosted
Main metric	end-to-end latency	throughput + cost
Failure risk	missed control deadline	failed / slow job

The ML workflow can be similar while the operating requirements are completely different.

Ask these questions in order

Design serving first; then placement, ownership and evidence.

Fresh inputs?	Reusable forecasts → batch; request-specific work → online.
Must caller wait?	No → consider a job and async completion.
No network?	Consider local or edge inference.
Who operates it?	Compare custom vs managed and total cost.
Can you prove it?	Observe identity, behavior, latency and failure.

Patterns coexist. These questions guide decisions; they are not a provider-specific recipe.

Five ideas worth keeping

Reliable serving is a chain of explicit, testable engineering choices.

Pattern	Choose for latency and freshness.
Axes	Separate computation, interaction and location.
Placement	Use network, hardware and operating constraints.
Operations	Measure capacity, startup delay and cost.
Evidence	Emit logs, metrics and runtime identity.

Continue to the course's CI material: checks that can detect and reject a bad deployment.

References & further reading

Public documentation and well-known references for the ideas in this lecture.

Containers	Docker documentation / run, logs, inspect
Config and logs	The Twelve-Factor App / config and logs
Serving design	Stanford CS329S
Inference modes	AWS SageMaker AI
Telemetry	OpenTelemetry concepts
Deployment	FastAPI deployment concepts

Check the current documentation before relying on product features, limits or pricing.